Introduction to Programming - Fall 2026

Dictionaries and sets

Look things up by name. Keep only one of each.

Store prices, stock and votes in dictionaries, and use sets to find unique values.

Part 1: Dictionaries

Find a value by its name instead of by its position.

01	Quick recap
02	Dictionaries are everywhere
03	Why not two lists?
04	Creating and reading
05	Changing a dictionary
06	Looping over a dictionary
07	Counting with a dictionary

Practice: a café menu and a shop's stock.

01 - Quick recap

A list keeps values in order. We find a value by its **index**.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])            # apple

def count_long(words):
    count = 0
    for word in words:
        if len(word) > 5:
            count += 1
    return count

print(count_long(fruits))   # 2
```

Today, we find values by a **name** instead of a number.

Quick recap: Functions

A function is a named piece of code that performs a task.

```
def total_price(price, quantity):
    return price * quantity

cost = total_price(3, 2)
print(cost)  # 6
```

- `def` defines a function; `price` and `quantity` are its **parameters**.
- `total_price(3, 2)` **calls** it with the arguments `3` and `2`.
- `return` gives the result back. `print()` displays it.

02 - Dictionaries are everywhere

Many things in daily life connect a **key** to a **value**.

Phone contacts

Ana → 0151 111

Shop price list

tea → 2 euros

A dictionary book

hello → a greeting

School report

maths → grade 2

You look up the key, and you get its value.

The same idea in Python

A Python **dictionary** stores these pairs.

```
contacts = {"Ana": "0151 111", "Ben": "0152 222"}
prices = {"tea": 2, "coffee": 3}
meanings = {"hello": "a greeting", "apple": "a round fruit"}
report = {"maths": 2, "English": 1}
```

```
print(prices["coffee"])     # 3
print(report["English"])    # 1
```

You will use dictionaries in almost every Python program you write.

03 - Why not two lists?

We could keep items and prices in two lists.

```
items = ["tea", "coffee", "cake"]
prices = [2, 3, 4]

position = items.index("coffee")
print(prices[position])     # 3
```

- We must keep both lists in the same order.
- Adding or removing an item means changing both lists.

A dictionary keeps each item **together with** its price:

```
menu = {"tea": 2, "coffee": 3, "cake": 4}
print(menu["coffee"])       # 3
```

04 - Creating a dictionary

```
menu = {"tea": 2, "coffee": 3, "cake": 4}
```

Part	Meaning
`{ }`	Start and end of the dictionary
`"tea"`	A **key**: the name we look up
`:`	Connects a key to its value
`2`	A **value**: what we get back
`,`	Separates the key-value pairs

Keys are often strings. Values can be anything: numbers, strings, lists…

An empty dictionary is `{}`.

Reading a value

Put the key in square brackets.

```
menu = {"tea": 2, "coffee": 3, "cake": 4}

print(menu["tea"])          # 2
print(menu["cake"])         # 4
print(menu["pizza"])        # crashes!
```

```
KeyError: 'pizza'
```

A **KeyError** means the key is not in the dictionary.

What will this print?

```
print(menu["Tea"])
```

Read safely

Check first with `in`:

```
if "pizza" in menu:
    print(menu["pizza"])
else:
    print("Not on the menu")
```

Or use `.get(key, default)`. It returns the default when the key is missing.

```
print(menu.get("tea", 0))     # 2
print(menu.get("pizza", 0))   # 0
```

`in` checks the **keys**, not the values:

```
print("tea" in menu)          # True
print(2 in menu)              # False
```

Exercise: Café menu · 5 min

You are building a café till. A dictionary stores each dish's price.

1. Make `menu` : soup costs 2, salad costs 3 and pasta costs 4.
2. Complete `price_of(menu, dish)` . **Return** the dish's price, or `0` if it is not on the menu. Print the prices of `"salad"` and `"pizza"` .
3. **(Advanced)** Write `order_total(menu, dishes)` to **return** the total for a list of dishes. Reuse `price_of` .

Expected input and output

```
price_of(menu, "salad")    # returns 3
price_of(menu, "pizza")    # returns 0
order_total(menu, ["soup", "pasta", "soup"])
# returns 8
```

Help

```
def price_of(menu, dish):
    pass

def double(number):
    return number * 2
```

Replace `pass` with your code.

05 - Changing a dictionary

Use `=` with a key. A new key is **added**; an existing key is **updated**.

```
menu = {"tea": 2, "coffee": 3}

menu["cake"] = 4       # add a new item
menu["tea"] = 3        # update the price of tea
print(menu)            # {'tea': 3, 'coffee': 3, 'cake': 4}
```

Remove a pair with `del`, and count pairs with `len`:

```
del menu["coffee"]
print(menu)            # {'tea': 3, 'cake': 4}
print(len(menu))       # 2
```

Each key appears **only once**. Setting it again replaces the old value.

A function can change a dictionary

When we give a dictionary to a function, the function can change it.

```
def raise_price(menu, item):
    menu[item] = menu[item] + 1

menu = {"tea": 2, "coffee": 3}
raise_price(menu, "tea")
print(menu)           # {'tea': 3, 'coffee': 3}
```

The function did not return anything, but `menu` changed. Lists work the same way: `append` inside a function changes the caller's list.

06 - Looping over a dictionary

Use `.keys()` to loop over a dictionary's **keys**.

```
menu = {"tea": 2, "coffee": 3, "cake": 4}

for key in menu.keys():
    print(key, menu[key])
```

```
tea 2
coffee 3
cake 4
```

Only the values: `menu.values()`

```
total = 0
for price in menu.values():
    total += price
print(total)          # 9
```

Keys and values together: `.items()`

`.items()` gives each pair as a tuple. We unpack it in the loop.

```
for item, price in menu.items():
    print(f"{item} costs {price} euros")
```

Part	Meaning
`menu.items()`	Every pair: `("tea", 2)`, `("coffee", 3)`, …
`item, price`	Unpacks each pair into two names

```
tea costs 2 euros
coffee costs 3 euros
cake costs 4 euros
```

The loop goes through the pairs in the order they were added.

A list of dictionaries

Each dictionary can describe one thing. A list can hold many of them.

```
students = [
    {"name": "Ana", "score": 18},
    {"name": "Ben", "score": 12},
]

for student in students:
    if student["score"] >= 15:
        print(student["name"], "passed")
```

```
Ana passed
```

`student` is one dictionary, so `student["name"]` reads its name.

Exercise: Shop stock · 15 min

You run a small corner shop. A dictionary stores how many of each item you have.

1. Write `restock(stock, item, amount)`. It adds `amount` to the item's count. A new item starts at `amount`. It returns nothing.
2. Write `sell(stock, item)`. If the item's count is above 0, take 1 away and **return** `True`. Otherwise **return** `False`.
3. **(Advanced)** Write `out_of_stock(stock)`. It **returns** a list of the items with a count of `0`.

Expected input and output

```
stock = {"apples": 5, "bread": 1, "milk": 0}
sell(stock, "bread")      # returns True
sell(stock, "bread")      # returns False
restock(stock, "eggs", 6)
print(stock)
# {'apples': 5, 'bread': 0, 'milk': 0, 'eggs': 6}
out_of_stock(stock)       # returns ['bread', 'milk']
```

Help

```
stock.get("eggs", 0)      # 0
for item, count in stock.items():
    print(item, count)
```

Starter: exercises/08-dictionaries-and-sets/shop-stock/

07 - Counting with a dictionary

How many times does each letter appear in `"banana"`?

```
counts = {}
for letter in "banana":
    counts[letter] = counts.get(letter, 0) + 1

print(counts)     # {'b': 1, 'a': 3, 'n': 2}
```

The key is the thing we count. The value is how many we have seen so far.

`counts.get(letter, 0)` gives the current count, or `0` the first time.

Trace the count

```
counts[letter] = counts.get(letter, 0) + 1
```

`letter`	`counts.get(letter, 0)`	`counts` afterwards
`b`	0	`{'b': 1}`
`a`	0	`{'b': 1, 'a': 1}`
`n`	0	`{'b': 1, 'a': 1, 'n': 1}`
`a`	1	`{'b': 1, 'a': 2, 'n': 1}`
`n`	1	`{'b': 1, 'a': 2, 'n': 2}`
`a`	2	`{'b': 1, 'a': 3, 'n': 2}`

Let's count the letters of your name together.

Break

15 minutes

Part 2: Sets

Keep one of each, and compare groups.

08	What is a set?
09	Using sets
10	Comparing sets
11	Hangman with a set
12	Choosing a collection
13	Homework and questions

Practice: count unique visitors and plan a party.

08 - What is a set?

A **set** is a collection where each value appears **only once**.

You already used one! In Hangman, `get_english_words_set` returned a set of words.

```
colours = {"red", "blue", "red", "green"}
print(colours)        # {'blue', 'green', 'red'}
print(len(colours))   # 3
```

- No duplicates: the second `"red"` is ignored.
- No order: the printed order may be different on your computer.
- No index: `colours[0]` does not work.

An empty set

`{}` is an empty **dictionary**, not a set.

```
empty_dict = {}
empty_set = set()

print(type(empty_dict))   # <class 'dict'>
print(type(empty_set))    # <class 'set'>
```

A set uses curly brackets with values only. A dictionary uses curly brackets with `key: value` pairs.

```
{"tea", "coffee"}         # a set
{"tea": 2, "coffee": 3}   # a dictionary
```

09 - Using sets

```
seen = set()
seen.add("Ana")
seen.add("Ben")
seen.add("Ana")       # already there, nothing changes

print(len(seen))      # 2
print("Ana" in seen)  # True

seen.remove("Ben")
print(seen)           # {'Ana'}
```

Code	What it does
`s.add(x)`	Adds `x` if it is not there yet
`x in s`	`True` if `x` is in the set
`s.remove(x)`	Removes `x` (crashes if it is missing)
`len(s)`	How many values

Remove duplicates from a list

`set(a_list)` makes a set with one of each value.

```
visits = ["Ana", "Ben", "Ana", "Cleo", "Ben"]
people = set(visits)

print(len(visits))    # 5 visits
print(len(people))    # 3 different people
```

A loop works on a set too:

```
for person in people:
    print("Welcome,", person)
```

Remember: the loop order is not guaranteed.

Exercise: Unique visitors · 5 min

A small museum writes down the name of every visitor at the door. Some people come more than once.

1. Write `unique_visitors(names)`. It **returns** how many different people visited. Use a set.
2. Print the result for `["Ana", "Ben", "Ana", "Cleo", "Ben"]`.
3. **(Advanced)** Write `first_visits(names)`. It **returns** a list of names in the order they first came, without duplicates. Use a set to remember who you have already seen.

Expected input and output

```
visits = ["Ana", "Ben", "Ana", "Cleo", "Ben"]
unique_visitors(visits)   # returns 3
first_visits(visits)
# returns ['Ana', 'Ben', 'Cleo']
```

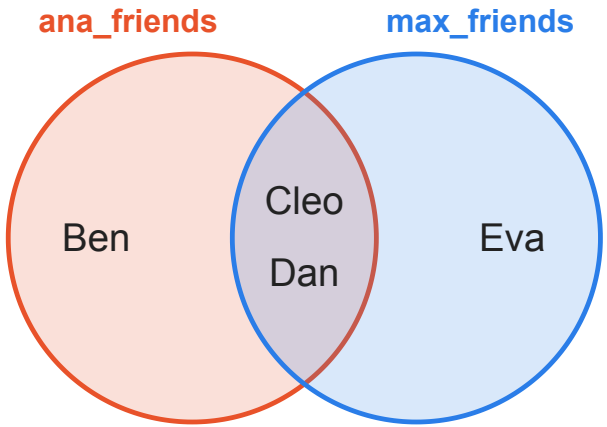
Help

```
seen = set()
seen.add("Ana")
print("Ana" in seen)      # True
```

Starter: exercises/08-dictionaries-and-sets/unique-visitors/

10 - Comparing sets

Ana and Max each have a set of friends.




```
ana_friends = {"Ben", "Cleo", "Dan"}
max_friends = {"Cleo", "Dan", "Eva"}
```

Union, intersection and difference

Method	Meaning	Result
`ana_friends.union(max_friends)`	In **either** set	`{'Ben', 'Cleo', 'Dan', 'Eva'}`
`ana_friends.intersection(max_friends)`	In **both** sets	`{'Cleo', 'Dan'}`
`ana_friends.difference(max_friends)`	In Ana's, **not** in Max's	`{'Ben'}`

Each method returns a **new** set. The original sets do not change.

Exercise: Party guests · 10 min

Ana and Max are throwing a party together. Each of them has a set of friends to invite.

1. Write `common_friends(friends1, friends2)`. It **returns** the set of friends that are in both sets.
2. Write `all_guests(friends1, friends2)`. It **returns** a set of everyone invited, each person once. Print how many guests are invited.
3. **(Advanced)** Write `everyone(groups)`. `groups` is a list of sets. It **returns** one set with every guest from every group. Use a loop.

Expected input and output

```
ana_friends = {"Ben", "Cleo", "Dan"}
max_friends = {"Cleo", "Dan", "Eva"}
common_friends(ana_friends, max_friends)
# returns {'Cleo', 'Dan'}
len(all_guests(ana_friends, max_friends))  # 4
everyone([{"Ben"}, {"Ben", "Cleo"}, {"Eva"}])
# returns {'Ben', 'Cleo', 'Eva'}
```

Help

```
a = {1, 2, 3}
b = {2, 3, 4}
a.union(b)          # {1, 2, 3, 4}
a.intersection(b)   # {2, 3}
```

Starter: exercises/08-dictionaries-and-sets/party-guests/

Shorter set operators

You have used the set methods. The same operations also have shorter forms:

Method	Operator	Meaning
`a.union(b)`	`a | b`	In either set
`a.intersection(b)`	`a & b`	In both sets
`a.difference(b)`	`a - b`	In `a`, but not in `b`

Both forms return a new set. The method names can be easier to read when you are learning.

11 - Hangman with a set

Last week, we stored the guessed letters in a list.

```
guessed_letters = []
guessed_letters.append("i")
```

A set fits better: each letter can only be guessed once, and the order does not matter.

```
guessed_letters = set()
guessed_letters.add("i")
guessed_letters.add("i")       # no duplicate

print("i" in guessed_letters)  # True
```

The rest of the game still works: `letter in guessed_letters` checks a set just like a list.

12 - Choosing a collection

	List	Tuple	Dictionary	Set
Looks like	`[1, 2]`	`(1, 2)`	`{"a": 1}`	`{1, 2}`
Keeps order	Yes	Yes	Yes	No
Duplicates	Allowed	Allowed	Keys unique	Not allowed
Can change	Yes	No	Yes	Yes
Find a value by	Index	Index	Key	`in` only

Which collection would you use?

Choose a list, tuple, dictionary or set.

1. The songs in a playlist, in order
2. Each country and its capital
3. The different words used in a book

13 - Homework

Five tasks. Each has an `exercise.py` and its own `check.py`.

Task	Practise
1. Vote counter	Counting with a dictionary
2. Phone book	Adding, reading and deleting entries
3. Word counter	Counting words, then a set of repeated words
4. Shopping cart project	Dictionaries and sets in a small shop program
5. Products by category	Collecting product names in lists inside a dictionary

Folder: exercises/08-dictionaries-and-sets/

Homework 1: Vote counter

Your class votes on where to go for a trip. Each vote is a place name in a list.

1. Write `count_votes(votes)`. It **returns** a dictionary: each place and its number of votes.
2. Print the result for `["zoo", "museum", "zoo", "park", "zoo", "park"]`.
3. **(Advanced)** Write `winner(counts)`. It **returns** the place with the most votes. Loop over `counts.items()`.

Expected input and output

```
votes = ["zoo", "museum", "zoo",
         "park", "zoo", "park"]
count_votes(votes)
# returns {'zoo': 3, 'museum': 1, 'park': 2}
winner({'zoo': 3, 'museum': 1, 'park': 2})
# returns 'zoo'
```

Help

```
counts = {}
counts["zoo"] = counts.get("zoo", 0) + 1
```

Folder: homework-01-vote-counter/

Homework 2: Phone book

You are building a phone book with names and numbers.

1. Write `add_contact(book, name, number)` to add or update a contact. It returns nothing.
2. Write `find_number(book, name)` to **return** the number, or `"Unknown"`.
3. Write `remove_contact(book, name)` to delete it and **return** `True`, or **return** `False` if missing.
4. **(Advanced)** Write `names_starting_with(book, letter)` to **return** a list of names starting with `letter`.

Expected input and output

```
book = {"Ana": "0151 111", "Ben": "0152 222"}
find_number(book, "Zoe")    # returns 'Unknown'
add_contact(book, "Zoe", "0153 333")
find_number(book, "Zoe")    # returns '0153 333'
remove_contact(book, "Ben") # returns True
remove_contact(book, "Ben") # returns False
names_starting_with(book, "Z")  # ['Zoe']
```

Help

```
del book["Ana"]
name = "Zoe"
print(name[0])          # Z
```

Folder: homework-02-phone-book/

Homework 3: Word counter

You are writing a tool that shows which words a text repeats.

1. Write `count_words(sentence)`. It **returns** a dictionary: each word and how many times it appears.
2. Print the result for `"the cat and the dog and the bird"`.
3. **(Advanced)** Write `repeated_words(sentence)`. It **returns** a set of the words that appear more than once.

   Reuse `count_words`.

Expected input and output

```
count_words("the cat and the dog")
# {'the': 2, 'cat': 1, 'and': 1, 'dog': 1}
repeated_words("the cat and the dog and the bird")
# returns {'the', 'and'}
```

Help: `split()` turns a sentence into a list of words.

```
words = "the cat sat".split()
print(words)    # ['the', 'cat', 'sat']
```

Folder: homework-03-word-counter/

Homework 4: Shopping cart project

Copy the code from the starter: homework-04-shopping-cart/exercise.py

You are building a shop. `PRICES` stores each price; the cart stores each quantity.

1. Write `add_to_cart(cart, item, quantity)`. Add the quantity and **return** `True`, or **return** `False` if the item is not in `PRICES`.
2. Write `cart_total(cart)` to **return** the total price of the cart.
3. **(Advanced)** Write `sale_total(cart, sale_items)` to **return** the total with half-price items from the set `sale_items`.

Expected input and output

```
cart = {}
add_to_cart(cart, "apple", 3)   # True
add_to_cart(cart, "apple", 2)   # cart: {'apple': 5}
add_to_cart(cart, "pizza", 1)   # False
add_to_cart(cart, "cheese", 1)  # True
cart_total(cart)                # 10
sale_total(cart, {"cheese"})    # 7.5
```

Help

```
PRICES["cheese"]          # 5
cart.get("apple", 0)      # 0 if not in the cart
for item, quantity in cart.items():
    print(item, quantity)
```

Homework 5: Products by category (Advanced)

Copy the code from the starter: homework-05-products-by-category/exercise.py

A shop has a list of products. Each has a name and a category.

1. Write `products_by_category(products)` to **return** a dictionary: each category is a key and its value is a list of product names, in their original order.
2. Print the result for `PRODUCTS`.
3. **(Advanced)** Write `categories_with_multiple(products)` to **return** the set of categories with more than one product. Reuse `products_by_category`.

Expected input and output

```
products_by_category(PRODUCTS)
# {'fruit': ['apple', 'banana'],
#  'bakery': ['bread', 'cake'],
#  'dairy': ['milk']}
categories_with_multiple(PRODUCTS)
# {'fruit', 'bakery'}  # order may differ
```

Help

```
product = {"name": "apple",
           "category": "fruit"}
product["category"]   # 'fruit'
names = []
names.append(product["name"])
```

The main ideas

- A **dictionary** connects keys to values: `{"tea": 2}`.
- Read with `d[key]`, or safely with `in` and `d.get(key, default)`.
- Add or update with `d[key] = value`; remove with `del d[key]`.
- Loop over pairs with `for key, value in d.items():`.
- Count things with `counts[x] = counts.get(x, 0) + 1`.
- A **set** keeps one of each value: `set(a_list)` removes duplicates.
- Compare sets with `union`, `intersection` and `difference`.

Questions?